Lessons Learned — CS6650 Final Project

Sampath Pranay Beela

Spring 2026

Where I started

When I wrote my midterm mastery, I said something like: before CS6650, I had no formal understanding of distributed systems. My backend experience was limited to REST APIs I shipped to Vercel and called done. I treated deployment as a configuration step that happened after coding, not as a design concern. ALBs, ECS tasks, horizontal scaling — those concepts were theoretical vocabulary I knew but had never had to reason about.

The midterm landed right after I built my first load-balanced ECS service. At the time it felt like the hardest thing I had built all year: pushing images to ECR, wiring an ALB target group, watching traffic fan out across task replicas, running Locust against it, and seeing throughput climb as I added replicas. I remember thinking that the mental leap from "one server" to "fleet behind a load balancer" was *the* thing this course would teach me.

Going into the final project, I thought the hard part was already behind me. I was wrong. The ECS/ALB/load-testing stack I built for the midterm turned out to be the *scaffolding*. The hard part of the final project was everything that happens inside the request path — concurrency, atomicity, idempotency, observability, failure recovery — and that is where almost every lesson I learned this semester actually lived.

From "deploying code" to "designing for failure"

The single biggest mindset shift happened during Experiment 2, the worker crash recovery test. My job was to submit a transfer and then kill the worker task while it was holding the message, and confirm no money was lost or duplicated. In theory I already knew this would work — the debit and credit were wrapped in a single DynamoDB `TransactWriteItems`, so the commit was atomic, and the worker only deleted the SQS message *after* the commit succeeded. So if the worker died, the message would come back after the visibility timeout, and some other worker would pick it up.

What I did not anticipate was how much it would change the way I read my own code. Before the experiment, I had written the worker loop thinking in terms of the happy path: receive message, process, delete. The crash test forced me to re-read every single line and ask "what if the worker dies right here?" Between the `ReceiveMessage` call and the `DeleteMessage` call there are five distinct places where the worker could crash, and the system has to be safe at every single one of them. It is only safe because (a) the commit is one atomic `TransactWriteItems`, (b) the commit's condition is `status = PENDING`, so a redelivered message whose predecessor already completed can't double-apply, and (c) the delete happens last. Remove any of those three and the system loses money somewhere.

In my midterm I wrote briefly about Two-Phase Commit and how modern distributed systems avoid it in favor of eventual consistency. The final project is the concrete version of that idea. We did not use 2PC across services; we used one atomic commit inside a single service (DynamoDB's transactional write), plus an idempotency layer at the API, plus SQS's at-least-once redelivery. That is three independently-reasoned mechanisms that compose to give exactly-once *financial effect*

under at-least-once *message delivery*. Reading about it in lecture is abstract. Watching `aws ecs stop-task` kill my worker and then seeing the next task resurrect the transfer is not.

Next time I build anything with money in it, the crash test is the first experiment I will run, not the last. It tells you more about your design than a throughput benchmark ever will.

Optimistic vs. pessimistic: tied throughput is not the whole story

Experiment 3 was the most intellectually honest experiment I ran. Going in, I had a strong prior: pessimistic locking is a heavier mechanism (it writes an extra row, takes a lease, coordinates between workers), so it should lose. I expected optimistic mode to win on throughput by a noticeable margin at the two user counts I was testing.

The numbers came back tied. Both modes hit 108–115 req/s at both contention levels. I stared at the summary for longer than I want to admit before I understood why. The bottleneck at this workload was not the locking mechanism; it was the DynamoDB partition. A hot account lives on one partition, and the partition has a fixed write ceiling. Both modes have to serialize writes through that ceiling, and so both modes get the same throughput. The locking mechanism was a decision about *how* they serialized, not *whether*.

Where the modes actually diverged was in the tail. Pessimistic's p95 was lower at 25 users because it never had to pay a retry backoff (100 ms, then 200 ms, then 400 ms). That insight —that the p95 difference was fundamentally about backoff, not about locks — was not obvious to me from the code. It only became obvious when I stared at the conflict-rate chart alongside the p95 chart, and realized the p95 gap was roughly the expected value of the first backoff step times the conflict rate.

The broader lesson, which I will carry into any future system: **throughput and latency are two different dials, and optimizing one does not automatically move the other.** A midterm-me would have looked at the tied throughput numbers and concluded "they're the same, doesn't matter." The final project taught me that "the same at average" often means "very different at p95", and if you care about tail latency (which real user-facing systems do), you have to measure both.

The bug I am most proud of catching

Midway through Experiment 4, the scaling sweep, I hit a transient balance mismatch. The first iteration (1:1) reported a $0.00 difference. The second iteration (2:2) reported $89.30 missing. My first reaction was panic: this was exactly the class of bug — a real money leak in a payment system — that the course had been warning me about all semester.

I spent about forty minutes in diagnostic mode. I scanned the DynamoDB accounts table directly, ran a Python script to identify which accounts were off-balance, looked at the transactions table for stuck `PENDING` records, and inspected the worker logs. By the time I finished the direct scan, the total came back to exactly $1,010,000. The money had reappeared.

The bug was not in the money-movement code. It was in the verification harness. Two compounding issues: (1) the script was running `verify_balances.py` before the SQS queue had fully drained — some transfers were still in flight — and (2) the verifier was using an *eventually-consistent* DynamoDB `Scan`, which during active writes can see one account's updated balance on one page of the scan and the same account's stale balance on another page. The invariant had never actually been violated. The scanner was just seeing a non-atomic snapshot.

I fixed both: added a queue-drain wait loop (poll `ApproximateNumberOfMessagesVisible` and

`...NotVisible` until both hit zero, plus a 15-second settle buffer), and flipped the verifier to `ConsistentRead=True`. The next run reported a $0.00 difference, and every run since has too.

The reason I am proud of this catch, in spite of the fact that it turned out to be a harness bug and not a system bug, is that I trained myself during the investigation to not trust my instinct. My instinct said "you lost money." The evidence said "you don't know yet — go read the data." Reading the data revealed a different story. In a payments context that discipline matters more than any single technical skill. The worst thing I could have done was ship a "fix" to the money-movement code in response to a bug that wasn't in the money-movement code.

This ties back to something Parnas wrote and the course emphasized: **information hiding is also information discipline**. The verifier was conflating two concerns (whether the system is correct, and whether the observer can see it being correct), and the conflation led me toward the wrong fix. Separating "system state" from "my view of system state" was the real fix.

What went wrong, and how I would handle it next time

A non-exhaustive list of the things I would do differently on day one if I could restart:

1. **Build for Linux/amd64 from day one.** My laptop is Apple Silicon. Fargate runs `linux/amd64`. My first ECS deployment failed with `CannotPullContainerError: image Manifest does not contain descriptor matching platform 'linux/amd64'`, and it took me a while to realize the problem was not in the image, the task definition, or the IAM role — it was in the architecture of the binary. I now know to reach for `docker buildx build --platform linux/amd64` the instant I target a cloud runtime, and I would add that flag to my very first Dockerfile next time rather than discovering it via a failing deployment.
2. **Treat the verification harness as first-class code.** I wrote the load-test and verification harnesses with a "we'll tidy them up later" mindset and paid for it exactly once, during the Experiment 4 false-positive. Next time I will write the harness with the same level of care as the service itself — strongly-consistent reads, drain waits, explicit warnings instead of fatal aborts — so a harness bug does not masquerade as a system bug.
3. **Build observability in, not on.** The worker's metrics endpoint and the periodic JSON `METRICS` log line both got added during Weeks 5–6, after I realized I had no way to compute conflict and retry rates from outside the worker. If I had built those counters into the worker on day one — before I wrote the first load test — I would have had a much better debugging picture throughout the project. The lesson generalizes: observability is a design concern, not a "we'll add dashboards later" concern.
4. **Macros and conventions for cross-OS scripts.** A single `$(date +%s%3N)` caused a whole Experiment 3 run to abort on macOS because macOS `date` has no `%N`. That kind of portability bug is entirely avoidable with a tiny convention: use Python for anything a bash-one-liner can't guarantee across platforms. Next time I will prefer `python3 -c` for any non-trivial arithmetic or time calculation inside a shell script.
5. **IAM assumptions need to be explicit.** The stack originally assumed AWS Academy's `LabRole` existed by default; it did not exist on my actual university account. I spent longer than I'd like figuring that out. Next time I will not assume any role exists — I'll write a one-shot bootstrap script on day one that creates the role if missing and attaches the exact managed policies I need (`AmazonECSTaskExecutionRolePolicy`, `AmazonDynamoDBFullAccess`, `AmazonSQSFullAccess`, `CloudWatchLogsFullAccess` turned out to be the minimum set). Every AWS project I build after this will ship with that bootstrap.

None of these are deep intellectual lessons. They are the operational scar tissue you only get by actually trying to deploy a real distributed system to a real cloud. Reading about Fargate is not the same as deploying to Fargate from a Mac.

Course concepts that clicked only after I used them

Three course concepts that I could quote on the midterm but did not truly understand until the final project:

Idempotency. In the abstract, idempotency means "doing it twice has the same effect as doing it once." In the final project, it meant five specific things: (1) the API checks if a transaction ID already exists before creating it, (2) the `PutTransactionIfNotExists` uses a conditional write so two API tasks racing the same ID cannot both win, (3) the worker short-circuits on terminal status before doing any balance work, (4) the TransactWriteItems commit has `ConditionExpression: status = PENDING` so a redelivered message cannot double-apply, and (5) the `MarkTransactionFailedIfPending` is itself conditional so it cannot overwrite a terminal state. Five mechanisms, each small, composing to one property. The course gave me the property; the project gave me the mechanisms.

Partial failure. On the midterm this was a sentence in my notes: "distributed systems fail partially." On the final project, partial failure was a specific worker task that got `SIGKILL`ed by ECS between `GetAccount` and `TransactWriteItems`, and the system had to survive it without my help. Designing for partial failure means every intermediate step has to be safe to abandon. That reframes every block of code from "what should this do" to "what happens if this never finishes."

Queue-based decoupling. The midterm taught me that a queue decouples producers from consumers. The final project taught me *why* that matters operationally: Experiment 4's 1-replica run had a p95 of 1,600 ms not because the system was slow, but because messages were piling up in SQS faster than one worker could drain them. The queue absorbed the backpressure that would otherwise have taken the API down. When I added the second worker, the queue drained, and p95 collapsed 13×. Without the queue, there is no place for backpressure to live; it has to land somewhere, and usually that somewhere is a failing API.

Closing

Day 1 of this course, I thought a distributed system was "an API that runs on more than one machine." I now think a distributed system is an exercise in designing for every failure you cannot prevent, so that the failures you cannot prevent do not become customer-facing problems. That reframing — from "make it work" to "make it survive" — is the single most valuable thing I am taking out of CS6650.

I am genuinely glad I built this project on real AWS rather than a local mock. The temptation was there; `docker-compose` would have been faster and cheaper. But none of the hardest lessons from this semester would have landed. Apple Silicon vs. amd64, LabRole vs. SSO, SQS visibility timeouts, DynamoDB eventual consistency on Scan, ECS task replacement — every one of those obstacles is something I will recognize immediately the next time I see it in production, and none of them exist in a local mock.

If I end up on a team that handles real money someday, the lesson I want to bring with me from this project is the one from Experiment 2: the first experiment you run on a payment system is not the benchmark. It is the crash test.